- RAG Learning

2.2 Programming Fundamentals

OCR GCSE Computer Science (J277) — Paper 2

2.2.1 Data Types and Casting

Every piece of data in a program has a **data type** that determines what kind of value it holds, how much memory it uses, and what operations can be performed on it. Using the correct data type is essential — trying to add two strings will concatenate them rather than performing arithmetic.

Type	Description	OCR Pseudocode	Python Example	When to Use
Integer	Whole number (positive, negative, or zero). No decimal point.	`score = 0`	`age = 17`	Counting, indexing, quantities that cannot be fractional (number of students, lives remaining)
Real (Float)	Number with a decimal point (including x.0). Can represent fractions.	`price = 3.99`	`pi = 3.14159`	Measurements, money, percentages, scientific calculations
Boolean	Only two possible values: `True` or `False`.	`gameOver = False`	`logged_in = True`	Flags, conditions, yes/no states, switches
Character	A single character (letter, digit, symbol, or space).	`grade = 'A'`	`initial = 'J'`	Menu choices, grades, single-character codes

© RAG Learning · Not for redistribution

String	A sequence of zero or more characters. Text data.	`name = "Jamie"`	`msg = "Hello"`	Names, messages, addresses, any text

Casting (Type Conversion)

Casting converts a value from one data type to another. This is essential because user input is always received as a string, so you must cast it to the correct type before performing calculations.

```
// OCR Pseudocode and Python casting functions:
int("5")       // String "5" → Integer 5
str(42)        // Integer 42 → String "42"
float("3.14")  // String "3.14" → Real 3.14
int(7.9)       // Real 7.9 → Integer 7 (TRUNCATES — does NOT round)
bool(1)        // Integer 1 → Boolean True
bool(0)        // Integer 0 → Boolean False

// WHY casting matters:
"5" + "3" = "53"     // String concatenation (not what we want!)
int("5") + int("3") = 8  // Arithmetic addition (correct!)

// Common use: reading numeric input in Python
age = int(input("Enter your age: "))
// input() returns a string; int() converts it to an integer
// Without int(), comparing age > 18 would fail or give wrong results

// Casting error: trying to cast non-numeric text to a number
int("hello")  // This causes a runtime ERROR (ValueError in Python)
// Always validate input before casting!
```

Exam Tip: A very common mistake is forgetting to cast input to a number before doing calculations. `input()` in Python always returns a string. If you need a number, wrap it in `int()` or `float()`. Also remember that `int()` **truncates** (cuts off decimals) rather than rounds: `int(7.9)` gives 7, not 8.

2.2.2 Variables, Constants, and Operators

Variables and Constants

- **Variable:** A named storage location in memory whose value **can change** during program execution. The name describes its purpose. E.g., `score = 0` then later `score = score + 10`.
- **Constant:** A named value that is set once and **cannot change** after that. Declared with `const` in OCR pseudocode. E.g., `const VAT_RATE = 0.2`. Constants make code more readable (the name explains the value) and easier to maintain (change the value in one place instead of everywhere). By convention, constant names use UPPER_CASE.
- **Local variable:** Declared inside a subroutine and exists only within that subroutine. Cannot be accessed from outside. Destroyed when the subroutine finishes. Preferred because it prevents unintended interference between subroutines.
- **Global variable:** Declared outside all subroutines and accessible from anywhere in the program. Persists for the entire program's lifetime. Should be avoided where possible because they make programs harder to debug (any part of the code could change the value unexpectedly) and harder to reuse (subroutines depend on external state).

```
// Demonstrating local vs global scope:

globalScore = 0                     // global variable

procedure addPoints(points)
    localBonus = points * 2         // local to this procedure
    globalScore = globalScore + localBonus
endprocedure

addPoints(5)
print(globalScore)                  // prints 10 — global is accessible
print(localBonus)                   // ERROR! localBonus does not exist here

# Python equivalent:
global_score = 0

def add_points(points):
    global global_score             # must declare global to modify it
    local_bonus = points * 2        # local variable
    global_score += local_bonus

add_points(5)
```

```
print(global_score)                 # 10
# print(local_bonus)               # NameError: not defined
```

Operators

Category	Operators	Examples and Explanation
Arithmetic	`+` `-` `*` `/` `MOD` `DIV` `^`	`7 MOD 3 = 1` (remainder after division). `7 DIV 3 = 2` (integer division, discards remainder). `2^8 = 256` (exponentiation). In Python: `%` for MOD, `//` for DIV, `**` for power.
Comparison (Relational)	`==` `!=` `<` `<=` `>` `>=`	`x == 5` (is x equal to 5?). `age >= 18` (is age at least 18?). Result is always Boolean (True or False). Note: `==` compares values; `=` assigns values. Confusing these is a very common bug.
Boolean / Logical	`AND` `OR` `NOT`	`x > 5 AND x < 10` (both must be true). `a OR b` (at least one must be true). `NOT loggedIn` (inverts the Boolean). In Python: `and`, `or`, `not` (lowercase).

MOD and DIV — Explained in Depth

MOD and DIV are frequently examined because they have many practical uses:

```
// MOD gives the REMAINDER after integer division:
17 MOD 5 = 2    (17 / 5 = 3 remainder 2 → MOD gives 2)
20 MOD 4 = 0    (20 / 4 = 5 remainder 0 → perfectly divisible!)
7 MOD 2 = 1     (7 / 2 = 3 remainder 1 → odd number)
8 MOD 2 = 0     (8 / 2 = 4 remainder 0 → even number)

// DIV gives the QUOTIENT (integer part, discarding remainder):
17 DIV 5 = 3    (17 / 5 = 3.4 → DIV gives 3)
7 DIV 2 = 3     (7 / 2 = 3.5 → DIV gives 3)

// Practical uses of MOD:
n MOD 2 == 0          // Check if n is even
n MOD 10              // Extract the last digit of n (1234 MOD 10 = 4)
seconds MOD 60        // Convert seconds to remaining seconds (125 MOD 60 = 5)
```

```
index MOD arrayLength // Wrap around (circular buffer/rotation)

// Practical uses of DIV:
n DIV 10              // Remove the last digit of n (1234 DIV 10 = 123)
seconds DIV 60        // Convert seconds to whole minutes (125 DIV 60 = 2)
(n DIV 10) MOD 10     // Extract the tens digit (1234: 123 MOD 10 = 3)
```

2.2.3 Sequence, Selection, and Iteration

Every program, no matter how complex, is built from just three fundamental constructs:

Construct	Description	When to Use
Sequence	Statements execute **one after another, top to bottom**, in the order they are written.	When steps must happen in a specific fixed order. Every program uses sequence.
Selection	The program **chooses** which code to execute based on a condition (IF/ELSE, SWITCH/CASE). Different paths through the code.	When the program must make a decision: "If this condition is true, do X; otherwise do Y."
Iteration (Loops)	A section of code **repeats** either a set number of times (FOR) or while/until a condition is met (WHILE, DO-UNTIL).	When actions need to be repeated. Avoids writing the same code multiple times.

Selection in Detail

```
// ── Simple IF ──────────────────────────────────────────
if temperature > 30 then
    print("It's hot!")
endif

# Python:
if temperature > 30:
    print("It's hot!")

// ── IF-ELSE ────────────────────────────────────────────
if age >= 18 then
```

```
    print("Adult")
else
    print("Minor")
endif

# Python:
if age >= 18:
    print("Adult")
else:
    print("Minor")

// ── IF-ELSEIF-ELSE (multiple conditions) ──────────────────
if score >= 70 then
    grade = "A"
elseif score >= 60 then
    grade = "B"
elseif score >= 50 then
    grade = "C"
else
    grade = "Fail"
endif

# Python:
if score >= 70:
    grade = "A"
elif score >= 60:
    grade = "B"
elif score >= 50:
    grade = "C"
else:
    grade = "Fail"

// ── SWITCH/CASE (test one variable against specific values) ─
switch day:
    case "Monday":
        print("Start of week")
    case "Friday":
        print("Nearly weekend")
    default:
        print("Midweek")
endswitch

# Python (match/case from Python 3.10, or use if/elif):
match day:
```

```
    case "Monday":
        print("Start of week")
    case "Friday":
        print("Nearly weekend")
    case _:
        print("Midweek")
```

Iteration (Loops) in Detail

```
// ── FOR loop (count-controlled) ──────────────────────────
// Use when you KNOW how many times the loop should run.
for i = 1 to 5
    print(i * i)        // prints: 1, 4, 9, 16, 25
next i

# Python:
for i in range(1, 6):   # range(1,6) gives 1,2,3,4,5
    print(i * i)

// ── WHILE loop (condition-controlled, pre-check) ──────────
// Use when you DON'T know how many times — may run 0 times.
// Condition checked BEFORE each iteration.
attempts = 0
while attempts < 3 AND password != "secret"
    password = input("Enter password: ")
    attempts = attempts + 1
endwhile

# Python:
attempts = 0
while attempts < 3 and password != "secret":
    password = input("Enter password: ")
    attempts += 1

// ── DO-UNTIL loop (post-check) ────────────────────────────
// Use when the body MUST run at least once.
// Condition checked AFTER each iteration.
do
    choice = input("Enter Y or N: ")
until choice == "Y" OR choice == "N"

# Python (no do-until; use while True with break):
```

```
while True:
    choice = input("Enter Y or N: ")
    if choice == "Y" or choice == "N":
        break
```

Exam Tip: Use **FOR** when the count is known in advance; **WHILE** when it is not (and the loop might not run at all); **DO-UNTIL** when the body must execute at least once (e.g., asking for input then validating). The key difference: WHILE checks *before* the loop body (may run 0 times); DO-UNTIL checks *after* (always runs at least once). In Python, `range(1, 11)` gives 1 to 10 (the upper bound is exclusive).

2.2.4 Arrays (1D and 2D)

An **array** is a fixed-size, ordered collection of elements of the **same data type**, stored in contiguous memory. Elements are accessed by their **index** (position number). OCR arrays are **zero-indexed** — the first element is at index 0, the last is at index (length - 1).

```
// ── 1D ARRAY ──────────────────────────────────────────────
array scores[5]           // declare array of 5 elements (indices 0-4)
scores[0] = 95            // first element
scores[1] = 72
scores[2] = 88
scores[3] = 41
scores[4] = 63            // last element (index 4, NOT 5!)

// Looping through all elements:
for i = 0 to 4
    print(scores[i])
next i

// Finding the maximum value:
max = scores[0]
for i = 1 to 4
    if scores[i] > max then
        max = scores[i]
    endif
```

```
next i
print("Highest score: " + str(max))

// Calculating the average:
total = 0
for i = 0 to 4
    total = total + scores[i]
next i
average = total / 5
print("Average: " + str(average))

# Python equivalent:
scores = [95, 72, 88, 41, 63]
for score in scores:
    print(score)
print("Highest:", max(scores))
print("Average:", sum(scores) / len(scores))

// ── 2D ARRAY (table/grid) ──────────────────────────────
array grid[3][4]           // 3 rows, 4 columns
grid[0][0] = "Alice"       // row 0, column 0
grid[0][1] = 17            // row 0, column 1
grid[1][0] = "Bob"
grid[1][1] = 16

// A 2D array is like a table/spreadsheet:
//          Col 0    Col 1    Col 2    Col 3
// Row 0:  "Alice"   17       "A"      95
// Row 1:  "Bob"     16       "B"      82
// Row 2:  "Carol"   17       "A"      91

// Nested loop to print all elements:
for row = 0 to 2
    for col = 0 to 3
        print(grid[row][col])
    next col
    print("")   // new line after each row
next row

# Python:
grid = [
    ["Alice", 17, "A", 95],
    ["Bob", 16, "B", 82],
    ["Carol", 17, "A", 91]
```

```
]
for row in grid:
    for item in row:
        print(item, end=" ")
    print()
```

2.2.5 String Manipulation

Operation	Description	OCR Pseudocode	Python	Result
Length	Number of characters	`len("Hello")`	`len("Hello")`	5
Substring	Extract part of string	`"Hello".substring(1,3)`	`"Hello"[1:4]`	"ell"
Upper	Convert to uppercase	`"hello".upper`	`"hello".upper()`	"HELLO"
Lower	Convert to lowercase	`"HELLO".lower`	`"HELLO".lower()`	"hello"
Concatenation	Join strings with `+`	`"Hi" + " " + "there"`	`"Hi" + " " + "there"`	"Hi there"
ASCII value	Character to number	`ASC('A')`	`ord('A')`	65
ASCII char	Number to character	`CHR(65)`	`chr(65)`	'A'

```
// Practical string manipulation example – reversing a string:
original = "Hello"
reversed = ""
for i = len(original) - 1 to 0 step -1
    reversed = reversed + original[i]
next i
```

```
print(reversed)    // "olleH"

# Python:
original = "Hello"
reversed_str = original[::-1]    # slicing shortcut
print(reversed_str)              # "olleH"

// Checking if a string is a palindrome:
function isPalindrome(word)
    word = word.lower
    reversed = ""
    for i = len(word) - 1 to 0 step -1
        reversed = reversed + word[i]
    next i
    return word == reversed
endfunction
```

2.2.6 File Handling, Records, SQL, and Random Numbers

File Handling

Programs need to save data permanently (beyond the program's runtime). File handling allows reading data from files and writing data to files on secondary storage.

```
// ── OCR Pseudocode ──────────────────────────────────────
// Reading from a file:
myFile = open("data.txt")
while NOT myFile.endOfFile()
    line = myFile.readLine()
    print(line)
endwhile
myFile.close()

// Writing to a file:
myFile = open("output.txt")
myFile.writeLine("Hello World")
myFile.writeLine("Second line")
myFile.close()

// ── Python ──────────────────────────────────────────────
```

```
// Reading:
with open("data.txt", "r") as f:    # "r" = read mode
    for line in f:
        print(line.strip())          # strip() removes trailing newline

// Writing (overwrites existing content):
with open("output.txt", "w") as f:  # "w" = write mode
    f.write("Hello World\n")
    f.write("Second line\n")

// Appending (adds to end of file):
with open("log.txt", "a") as f:     # "a" = append mode
    f.write("New entry\n")
```

The `with` statement in Python automatically closes the file when the block ends, even if an error occurs. Always close files when done to prevent data corruption.

Records

A **record** is a collection of related fields of **different data types** grouped together (unlike an array, which stores elements of the same type). Each field has a name and a data type.

```
// OCR Pseudocode:
record Student
    name : string
    age : integer
    grade : character
    average : real
endrecord

pupil1 = new Student()
pupil1.name = "Alice"
pupil1.age = 17
pupil1.grade = 'A'
pupil1.average = 91.5

// Array of records:
array classList[30] of Student
classList[0].name = "Alice"
classList[0].age = 17
```

SQL (Structured Query Language)

SQL is used to query (search and retrieve) data from databases. You need to know SELECT, FROM, WHERE, and ORDER BY.

```
// Basic SELECT query:
SELECT name, age FROM students WHERE age > 16

// Select ALL columns:
SELECT * FROM products WHERE price < 10.00

// Multiple conditions with AND / OR:
SELECT * FROM products WHERE price < 10.00 AND category = 'Electronics'
SELECT name FROM students WHERE year = 11 OR year = 12

// Sorting results:
SELECT name, score FROM results WHERE score >= 50 ORDER BY score DESC
// ASC = ascending (A-Z, 0-9) — default if not specified
// DESC = descending (Z-A, 9-0)

// Key SQL clauses explained:
// SELECT — which columns to return (* means all columns)
// FROM   — which table to query
// WHERE  — filter rows by condition (only rows matching are returned)
// ORDER BY — sort the results
```

Exam Tip: SQL keywords (SELECT, FROM, WHERE, ORDER BY) are conventionally written in UPPERCASE but the exam accepts lowercase. Remember: SELECT picks **columns**; WHERE filters **rows**. Use AND/OR to combine conditions. Use single quotes around text values in WHERE clauses (e.g., `category = 'Electronics'`). Numbers do not need quotes.

Random Numbers

```
// OCR Pseudocode:
x = random(1, 6)        // random integer between 1 and 6 inclusive (dice roll)

# Python:
import random
x = random.randint(1, 6)     # random integer 1 to 6 inclusive
```

```
y = random.random()           # random float 0.0 to < 1.0
z = random.choice(["A","B","C"])  # random element from a list
```

2.2.7 Subroutines — Procedures and Functions

A **subroutine** is a named block of code that performs a specific task and can be called (invoked) from anywhere in the program. Subroutines come in two types: **procedures** (which perform an action but do not return a value) and **functions** (which perform a calculation and return a result).

Feature	Procedure	Function
Returns a value?	**No** — performs an action (printing, saving, modifying) but does not give a value back	**Yes** — calculates something and `return`s the result to the caller
How called	Called as a standalone statement: `greet("Jamie")`	Called within an expression or assignment: `result = square(5)`

OCR Pseudocode	`procedure greet(name)`
	`  print("Hello, " + name)`
	`endprocedure`
		`function square(n)`
		`  return n * n`
		`endfunction`

Python	`def greet(name):` `  print("Hello,", name)`	`def square(n):` `  return n * n`

Parameters and Arguments

- **Parameter:** The variable name in the subroutine **definition**. It is a placeholder that will receive a value when the subroutine is called. E.g., `name` in `procedure greet(name)`.
- **Argument:** The actual value passed when the subroutine is **called**. E.g., `"Jamie"` in `greet("Jamie")`. The argument is assigned to the parameter.

```
// Multiple parameters example:
function calculateArea(length, width)
    return length * width
endfunction

area = calculateArea(5, 3)     // arguments 5 and 3
print(area)                     // prints 15

# Python:
def calculate_area(length, width):
    return length * width

area = calculate_area(5, 3)
print(area)   # 15
```

Benefits of Using Subroutines

- **Code reuse:** Write once, call many times from different places. No copy-paste duplication. If you need the same calculation in 10 places, write one function and call it 10 times.
- **Easier to debug:** Each subroutine can be tested independently in isolation (unit testing). If there is a bug, you only need to fix it in one place.
- **Improved readability:** The main program reads like a high-level overview; details are in subroutines. `calculateTax(income)` is immediately understandable.
- **Team development:** Different programmers can work on different subroutines simultaneously without interfering with each other.
- **Easier maintenance:** Changing the subroutine's implementation automatically updates everywhere it is called. If the tax rate changes, update one function.
- **Decomposition:** Subroutines are the practical implementation of decomposition — breaking a large problem into smaller, manageable sub-problems.

> **Exam Tip:** **Procedure = no return value** (it does something but gives nothing back). **Function = returns a value** (it calculates something and gives the result back). This is the most important distinction. Arrays are **zero-indexed** (first element at index 0, last at index length-1). A common off-by-one error is using `length` instead of `length-1` as the last index. For SQL: SELECT picks columns, FROM names the table, WHERE filters rows.